

Study Notes Series

2026 Spring

吴恩达机器学习笔记 (5)

课程：机器学习 作者：C++ 与草稿纸

本节摘要

本周讨论如何训练第四周定义的神经网络：用代价函数衡量输出误差，用反向传播高效计算每个权重的梯度，再用梯度下降或高级优化器更新参数。还介绍展开参数、梯度检验、随机初始化和完整训练流程。

- 多类神经网络使用交叉熵代价，并对非偏置权重进行正则化；
- 反向传播从输出层向输入层传播误差，得到每个 $\Theta^{(l)}$ 的梯度；
- 梯度检验用数值导数核对反向传播实现，随机初始化打破对称性；
- 自动驾驶等系统通常由感知、预测和控制等多个学习模块组成。

1 神经网络的代价函数

多类分类代价

设网络有 K 个输出单元，样本标签用 one-hot 向量 $\mathbf{y}^{(i)}$ 表示，输出为 $\mathbf{a}^{(L,i)}$ 。多类交叉熵代价可写为

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left[y_k^{(i)} \log a_k^{(L,i)} + (1 - y_k^{(i)}) \log(1 - a_k^{(L,i)}) \right].$$

对所有训练样本和输出单元求平均，模型训练就是寻找使 $J(\Theta)$ 最小的权重矩阵集合。

正则化神经网络

为抑制过拟合，在代价函数中加入除偏置列以外的权重平方和：

$$J_{\text{reg}}(\Theta) = J(\Theta) + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_i \sum_{j=1}^{s_{l+1}} \left(\Theta_{ji}^{(l)} \right)^2.$$

第一列通常对应偏置权重，不参与正则化。 λ 越大，模型越倾向于使用较小的权重。

2 反向传播算法

误差项

对第 i 个样本，记第 l 层的误差为 $\delta^{(l)}$ 。输出层误差由预测与标签之差给出：

$$\delta^{(L)} = \mathbf{a}^{(L)} - \mathbf{y}.$$

对隐藏层，误差从下一层反向传播：

$$\delta^{(l)} = ((\Theta^{(l)})^T \delta^{(l+1)}) \odot g'(\mathbf{z}^{(l)}), \quad l = L-1, \dots, 2,$$

其中 $\odot$ 表示逐元素乘法。偏置单元对应的误差不参与下一层传播。

反向传播流程

对每个训练样本执行：

1. 前向传播，保存每层 $\mathbf{z}^{(l)}$ 和 $\mathbf{a}^{(l)}$ ；
2. 计算输出层误差 $\delta^{(L)}$ ；
3. 从 $L-1$ 层向第 2 层反向计算误差；
4. 累加梯度：

$$\Delta^{(l)} \leftarrow \Delta^{(l)} + \delta^{(l+1)} (\mathbf{a}^{(l)})^T;$$

5. 取平均并加上正则化项，得到 $\partial J / \partial \Theta^{(l)}$ 。

反向传播的意义是利用链式法则共享中间结果，避免对每个参数重复计算完整的数值导数。

Sigmoid 导数

若 $g(z) = 1/(1 + e^{-z})$ ，则

$$g'(z) = g(z)(1 - g(z)).$$

因此保存 $a^{(l)} = g(z^{(l)})$ 后，可以直接用 $a^{(l)} \odot (1 - a^{(l)})$ 计算 Sigmoid 的导数。

3 实现与调试

3.1 展开参数

把矩阵参数变成向量

数值优化器通常要求参数是一个长向量。实现时把所有 $\Theta^{(l)}$ 按固定顺序拼接成 θ ，计算代价后再恢复为矩阵。展开和恢复必须使用完全一致的顺序，否则代价和梯度会对应到错误的参数。

3.2 梯度检验

数值梯度检查

对第 j 个参数，用中心差分近似导数：

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\theta + \varepsilon \mathbf{e}_j) - J(\theta - \varepsilon \mathbf{e}_j)}{2\varepsilon}.$$

将该数值结果与反向传播的解析梯度逐项比较。梯度检验只用于小规模调试，确认正确后必须关闭，否则计算成本会非常高。

反向传播的常见错误

- 把偏置列也加入正则化；
- 忘记误差传播中的转置或逐元素乘法；
- 梯度检验时仍使用随机变化的参数、数据或 dropout；
- 只检查代价不检查梯度，导致优化器无法正常下降；
- 用过大的学习率，使代价函数发散。

3.3 随机初始化与完整流程

随机初始化的作用

如果所有权重都初始化为相同值，同一层的神经元会得到完全相同的梯度，训练后仍然保持相同功能，这称为对称性问题。通常从小范围均匀分布中随机初始化权重，例如

$$\Theta_{ij}^{(l)} \sim U(-\epsilon, \epsilon).$$

初始化范围应与上一层和下一层的规模相称。

训练神经网络的工程流程

确定层数和每层单元数，随机初始化参数；实现前向传播和代价函数；实现反向传播得到梯度；用梯度检验验证实现；再调用优化器最小化正则化代价；最后在验证集和测试集上检查泛化能力。

4 应用：自动驾驶

从感知到控制

自动驾驶系统可以把摄像头图像、雷达和其他传感器作为输入，通过神经网络识别道路、车辆、行人和车道线，再由规划与控制模块决定转向、加速和制动。实际工程中常将系统拆为多个可单独评估的模块，并用大量覆盖边界情况的数据验证每个模块。

5 本周知识脉络

从前向传播到反向传播

前向传播计算预测，代价函数衡量预测与标签的差异；反向传播利用链式法则计算所有权重的梯度；正则化控制模型复杂度；梯度检验、随机初始化和分模块评估保证实现可以可靠训练。下一步应结合偏差、方差和验证集选择网络结构与正则化强度。